DESIGNING A UNIVERSAL INFORMATION LANGUAGE

OSCAR BENDER-STONE

ABSTRACT. Welkin is a formalized information language. We introduce its use cases and rigorously define its syntax and semantics. From there, we introduce the bootstrap, making Welkin completely self-contained. [TODO: determine how to phrase soundness and incompleteness. Should we include these?].

CONTENTS

1. Introduction 1
2. Rationale 5
   2.1. General Requirements 5
   2.2. Units 5
   2.3. Mechanizing Information 6
   2.4. Base Rules 6
   2.5. Base Operations 7
   2.6. Bootstrapping 8
3. Foundations 9
   3.1. Words and Handles 9
   3.2. Units 10
   3.3. Turing Completeness 12
   3.4. Queries and Information 14
4. Syntax 14
   4.1. Encoding 15
   4.2. Invertible Syntax Descriptions 17
   4.3. Strings and IDs 17
   4.4. The Welkin Grammar 18
   4.5. Validation and Transformations 19
   4.6. Proof of Unambiguity 20
5. Metatheory 22
   5.1. Establishing the Floor: $I\Delta_0$ 23
   5.2. Artemov's Approach 25
   5.3. Truth and Soundness 26
   5.4. The Meta Unit 27
   5.5. Proof of Complete Expressivity 27
6. Conclusion 29
   6.1. Comparisons to Existing Literature 29
   6.2. Future Work 29
References 31

DRAFT

1. INTRODUCTION

[TODO[MEDIUM]: determine if the phrase: "see Section ?" should be used, or "refer to" instead, to fully include all readers.]

Information Management is the study of systematically storing and organizing data. Since the beginning of humanity, there has been a massive influx of data, including physical and digital forms. Not a single person understands all of it. For effective use,

---

This thesis used Google Gemini (https://gemini.google.com/) for the following tasks: reviewing the iterature, discussing conjectures, clarifying new concepts, evaluating new language features, and creating templates for tables and figures. All ideas presented come from the author.

Date: March 6, 2026.

raw databases are increasingly structured into organized *information bases* to prove context surrounding the source and nature of data. The conceptual foundation for this process often traces back to Ackoff [Ack89]. His data-to-wisdom hierarchy posits that information is the process of transforming raw data into structured content. In turn, this content can be used to answer user queries. However, the term "information" has not been standardized in the literature and is used with distinct meanings. For example, Buckland [Buc91] recognizes three different ways information is interpreted: as a *process*, as *knowledge*, and as *a thing*. While each of these are important, Buckland emphasizes that to *use* information, one must work with information as a thing—a tangible record like a document or digital file. Another author, Floridi, builds upon this tangible idea by requiring data to be truthful [Flo10]. Information must not only be in concrete records, but also accurately represent a topic of interest. As shown in this small sample of authors, the word information has several meanings, and they are driven by the goal to organize data.

Despite efforts to understand information, there are ongoing problems in real world projects. One of these issues is the *depth* of domains. For example, at major retailers like Amazon, inventory and recommendation systems use immense amounts of data to power personalized recommendations to users [SL17]. Another issue is the immense *breadth* of different subjects. As a well known example, Wikipedia struggles to constantly update its vast and interconnected web of knowledge, as found in [Mes+15]. Along with the depth and breadth of different disciplines, standardization is complex. Most systems have their own unique data formats, so making formats that are **interoperable** is hard. (A format is **interoperable** if data can be transferred to different formats.) For instance, transferring information between two medical platforms is notoriously difficult, even if they store the *same* information about a patient [Rei17]. These problems continue to be a challenge in effectively utilizing frameworks for information.

In consideration of these issues, several solutions partially resolve some of them, but not completely. The following provides a small list of major existing solutions. Reading these is optional and not required to understand this thesis [TODO: rephrase this sentence?]. We will not revisit these until the conclusion, see Section 6.1.

- **Resource Description Framework (RDF):** These systems serve as the primary implementations of an internet standard called the Web Ontology Language (OWL) [Hit+09]. Information is stored as triples $\text{subject} - \text{predicate} \rightarrow \text{object}$. From there, the language can enforce rules and ensure new relationships are derived *only* from from previous ones. OWL has been highly successful in providing a machine-readable format to websites across the internet, see [Hit21]. However, despite their widespread adoption, RDF-based solutions struggle with extensive sources of data. Enforcing logical rules can be expensive [Zhu+23]. Additionally, because OWL treats missing sources of data as *unknown* rather than *false*, it often fails to resolve conflicts between two sources [Hit21].
- **Labeled Property Graphs (LPGs):** This is an architecture defined by the ability to store rich, internal metadata within its components. Building on a network, consisting of nodes and edges between them, LPGs enable both nodes (entities) and edges (relationships) to possess a set of key-value pairs known as *properties* [Ang18]. The internal metadata allows systems to efficiently traverse

Draft

networks, as attributes can be filtered and weighed on a single query pass. Due to their operational efficiency, LPGs are the primary choice in large scale commercial products, particularly real-time fraud detection tools and recommendation systems [RWE15]. Although LPGs are widely used, they are siloed by the lack of standards, making interoperability between databases challenging. Furthermore, LPGs lack an extensive logical engine, resulting in external tooling or complex query scripts [RWE15].

- **Cyc:** Designed to encompass broad subjects, Cyc is the largest proprietary database of knowledge, or *knowledge base*, known to date [Len95]. It achieves reliable queries by providing a logical engine, based on millions of hard coded "common sense" rules. However, despite its impressive scope, it relies on human experts to manually encode every rule. This limits how well the database scales [Har07]. Another issue is the absence of an active open standard, a result of the publicly available version, OpenCyc, being discontinued in 2017 [KBp].
- **fcaR (Formal Concept Analysis in R):** This tool is a modern implementation for the field of Formal Concept Analysis (FCA). FCA studies relationships through *formal concepts*, defined as a pair: an *extension* and an *intension*. Concepts are studied through a *concept lattice*, a hierarchical data structure that maps objects to attributes in a fixed domain. A modern implementation of this system is fcaR, a program developed in the R programming language. Given a fixed domain, the program can automatically identify the underlying core concepts and derived concepts [MDV26]. While being highly effective in specialized domains, concept lattices can become impractically large as the number of attributes increases [HS13]. This makes it difficult to interpret results when applied to large-scale databases or when bridging disparate domains [HS13].

This thesis designs a language to resolve these issues for Information Management. I call this language **Welkin**, based on an old German word meaning cloud [Dic25]. This word represents the expansive nature of the language, able to express any concept and justification that can be processed through a computer program.

The core result of this thesis is that Welkin fulfills two goals: it is **universal** and **scalable**. Moreover, Welkin has another goal: to be fully **standardized**. While many parts of this thesis partially reach this goal, the rest is left for a future work. More details are available in Table 1. The fundamental building block is based on **representations**, containing a relationship between a **sign** representing a **referent** in a **context**.[1] Contexts provide flexibility to define domain-specific terminology, and are provided rules inspired by McCarthy's work in AI systems [McC93]. Moreover, truth is defined relative to contexts and enables the user to recognize *under which conditions* a statement is true. In this way, the information base keeps theories with conflicting claims isolated [maybe different word than isolated?], providing a clear separation from other contexts.

[TODO: revise with latest version of what information is!] Building on representations, information is defined as a *collection* . preserved under equivalences relative to a

[1]There are similarities with representations and Peirce's semiotics. This theory studies the relationship between a symbol, the object it represents, and the interpretation that provides that meaning [Atk23]. Our notion is different in that contexts are a generalized interpretant, see Section 2.2.

DRAFT

context. To make the information base scalable, the base operations are kept as simple as possible. However, they are proven to be maximally expressive. The simplicity of the language aids in standardization as well. However, to keep this thesis self-contained, we will discuss standardization at a high level. The full standardization and bootstrap will be left for another work, see Section 6.

[TODO: add links to each cell]

Goal	Name	Description
Goal 1	**Universality**	The language must enable any user-created parameters, whose symbolic representation is accepted a computable function. Every computable function must be definable in the language.
Goal 2	**Scalability**	The database must appropriately scale to broad representations of information. Local queries must be efficient. Additionally, any future, optimizations that correctly implement the language should be expressible in the language.
Goal 3	**Standardization**	The language needs a rigorous and formal specification. Moreover, the bootstrap must be formalized. The grammar and bootstrap must be fixed to ensure complete forwards and backwards compatibility. Certificates must be reliably checked, with a transparent way to view every step of a proof.

TABLE 1. Goals for the Welkin language.

DRAFT

This thesis is organized according to Table 2.

Section Number	Title	Description
Section 2	**Rationale**	Introduces Welkin at a high level, with guiding examples.
Section 3	**Foundations**	Introduces the core theory behind Welkin, which is the starting point to specify the user-facing language.
Section 4	**Syntax**	Provides the grammar and proof that it is unambiguous.
Section 5	**Metatheory**	Proves that Welkin can express any proof.
Section 6	**Conclusion**	Concludes with comparisons to existing theories and possible applications, particularly in programming languages and broader academic knowledge management.

TABLE 2. Organization for the thesis.

2. Rationale

In this section, we justify the design of Welkin.

2.1. General Requirements.

The main purpose of an information base is to store information and answer user queries based on available information. To provide an upper bound on our requirements, we require two important properties:

1) The information base must be mechanized, which means only "effective operations" are allowed. We make "effective operations" precise with the technical notion of *partial computable functions*. Partial computable functions are computer programs that can process a finite input, and that either stop on an input or continue forever. The bedrock for these functions is the notion of a *Turing machine*, which models every current form of computation. For more information, see [Tur36]. To obtain universality, we need to express *every* one of these machines.
2) The information base must provide a mechanism to *ground* symbols defined by users. This stems from the Symbol Grounding Problem, posed by Harnard [Har90]. To avoid storing physical objects or such, users should have a way to provide *meaning* to symbols, or a way to *represent* their topic of interest.

Combined together, we can now state universality as follows: *an information base is universal if it can represent anything representable by a partial computable function.* We will clarify how we define "representable" in the next section.

2.2. Units.

[TODO[MEDIUM]: double check all examples!]

The first core concept in Welkin is the notion of a **handle**. Handles are left as free parameters in the theory[2] In an information base, handles are uniquely identified through their **ID**.

Example 2.2.0. In a scientific experiment, a handle could be an observation or experimental data. The unit is then written as a symbol, say u, and is *implicitly* bound to this meaning. To distinguish from other symbols, say v, the computational content is analyzed.[TODO[MEDIUM]: expand out this example!] △

Indexing handles is only one important aspect of handles. Another is that handles are precisely by their relationships. These are written as **representations** $a - c \to b$, which means that a represents b in context c. Here, we call a the **sign**, b the **referent**, and c the **context**. One can interpret this characterization as follows: handles are defined by how they are and are *not* restricted. This idea is directly inspired from Fine's idea of arbitrary objects [Fin85], and allows refinements to a study of interest.

[TODO: come up with a better example here! Maybe math based, but simple?]

[2]For mathematicians, this is analogous to how key notions are left undefined in Hilbert's formalism of geometry, or how a "set" is an undefined notion in set theory.

DRAFT

Example 2.2.1. A more looser example is a user written journal for therapy sessions, containing information about daily habits and emotions. While neither of these are stored in the information base, their handles are, via units habit and $\mathrm{emotions}$ in a context $\mathrm{journal}$. Moreover, multiple revisions of the journal can be made with dates or other unique IDs. △

We represent handles in Welkin using an extended notion called a **unit**. A unit is either a handle, a finite collection of units, or a representation. Collections of units can have a uniquely generated name by the information base called a **block**.[3]

Example 2.2.2.

△

Units define their own **context**, which means the unit provides its own names and representations. Writing $a - c \to b$, therefore, only applies to the *fixed* context c.

[TODO[SMALL]: use better names for entities/businesses/etc]

Example 2.2.3. A business could represent their operations using a unit $\mathrm{business}$ that contains units for their workers and ledgers. This allows another unit, say $\mathrm{business2}$, to contain its *own* label $\mathrm{workers}$ that is separate from the one in $\mathrm{business}$. In addition to separate labels, these contexts can have *distinct* rules, such as those for how business operations are performed. △

2.3. Mechanizing Information.

From the notion of a unit, we practically define information on a unit v as a *unit that tracks reductions in* v. Formally, we define information as a proof used in derivations, which include refinements of v to other units. Key to this definition is making it *context-based*, thereby enabling separate truth-management systems (those systems defined by some Turing machine). This notion corresponds to Burgin's idea of information as an *operator* that transforms a system, and is closely to Bateson's famous quote that "information is a difference that makes a difference" [Bat00]. Our practical distinction between information and knowledge is that we *use* information, but users can assert their own notions of these terms by creating restricted contexts. There are technical constructions to show that our definition is *as general* as possible (Definition 3.4.15).

2.4. Base Rules.

There are two primary inference rules, which we informally describe now:

- **Internal Transitivity:** if u represents v in context c and v represents x in context c, then u represents x in context c.
- **Lifting:** if u represents v in context c and p represents q in context v, then within context c, p represents q in *nested* context u.

Transitivity is a common axiom in many systems. It is particularly efficient when used in contexts, as shown by several projects [Len+90], [GWW07]. Lifting is a notion closely tied to McCarthy's notion of lifting, but formulated with representations. The idea is, if a represents b in context c, then a provides an *abstraction* for b relative to c. Wherever b is involved in a relationship, a can be used *instead*, acting as a proxy relative

[3] In programming languages, this is analogous to an anonymous function.

DRAFT

to c. Contexts provide an import mechanism as well, thereby providing Welkin with a module based system. We provide a simplified version of one McCarthy's examples below. For more existing examples in the literature, refer to [BF95].

[TODO: clean up further!]

Example 2.4.4. (McCarthy's Above Theory). Consider a set of physical blocks. Denote the unit above to represent the relation: *block* A *is above block* B. We want to say this could be on to mean: *block* A *is on block* B, or $\mathrm{floating_above}$ to mean: "block A is floating above block A. One way to state is as two axioms: . Alternatively, we could state as one axiom: $\mathrm{A} - \mathrm{on} \to \mathrm{B}$ implies $\mathrm{A} - \mathrm{above} \to \mathrm{B}$ and $\mathrm{A} - \mathrm{floating_above} \to \mathrm{B}$ implies $\mathrm{A} - \mathrm{floating_above} \to \mathrm{B}$. Lifting provides a more economical approach *without* needing to provide explicit terms, or provides a "point-free" rendering. We could that, within above, above represents on and above represents $\mathrm{floating_above}$. In other words, above *precisely abstracts* from the precise relationship between the two blocks. △

Another example concerns contexts in the presence of unique objects.

Example 2.4.5. Consider a biological survey of home pets, denoted by context Survey. Suppose there are two units in Mammal, say a dog Fido and a cat Lucy. We could state that Fido *represents* Mammal, to say that Fido acts as a stand-in for a mammal in the survey. The same could be said with Lucy representing Mammal. Note that, depending on the context, the referent need *not* be more refined than the sign; this is an intenional design choice for flexibility. With a another context $\mathrm{Taxonomy}$, we may naturally state that $\mathrm{Mammal} - \mathrm{Taxonomy} \to \mathrm{Animal}$, and we could say $\mathrm{Survey} - \mathrm{instanceof} \to \mathrm{Taxonomy}$. Thus, the lifting rule implies in context $\mathrm{instanceof}$, $\mathrm{Mammal} - \mathrm{Survey} \to \mathrm{Animal}$. Using transitivity, we obtain $\mathrm{Fido} - \mathrm{Survey} \to \mathrm{Animal}$ and $\mathrm{Lucy} - \mathrm{Survey} \to \mathrm{Animal}$ in context $\mathrm{instanceof}$. We can interpret this example as saying that the *relationships* of a general taxonomy are witnessed by a specific taxonomy, namely Survey, and that they propagate through contexts.[4] △

The remaining rules are primarily for efficiency, enabling users to have positionally invariant information, see Definition 3.2.11.

2.5. **Base Operations.**

Now, units and information themselves could be expressed in infinitely many languages, with slightly different syntax or semantics. Welkin is carefully designed to be a *minimal* expression of these concepts, with minimal friction to express any other universal information base. We write these in ASCII in `type-writer` font to show that they are shown digitally. These include:

- In ASCII, `a - b -> c` for representations. These can be interpreted *as* rewrite rules, depending on the context.
- Traditional braces `{ }` to denote closed definitions of contexts, inspired by the C programming language.

[4] In programming languages, particularly C++ [Str13] and Java [AGH05], this property is known as "upcasting", with a less refined base class being replaced with a more refined subclass. The reverse is "downcasting". Both are supported by Welkin for full expressivity.

- Paths via dots `.` that is inspired by the Python programming language. Relative paths are denoted with multiple dots `...`, and absolute imports are prefixed with `#`. Subsets of units can be written via `u.{v, x}`, or even `u.{v --> x}` to refer to a subset of representations.
- Imports are done through `@u`, which takes all subunits of `u` and puts them into the current scope. In other words, the implementation *implicitly* adds denotations `v <--> a.v` for each subunit `v` of `u`.
- Selecting specific nested units can be done via `u.{v, x}` or, equivalently, `@u.{v, x}`.
- Imports can be *negated* via the notation `{@g, ~u}`. This is an uncommon feature of most programming languages, appearing primarily in Haskell and CSS. While potentially opaque, Welkin provides robust definitions to ensure that negated forms can be easily translated into more explicit ones *and* back again. [TODO[SMALL]: provide link, probably to bootstrap or so?]
- Comments *are* strings can be treated as any unit. No comments need to be removed in the files and can *enhance* the study of new subjects.[5] These units can *then* be analyzed by others to promote translations to other human languages, or be studied through the *overarching relationships* in the units.

In general, the minimal restricted keywords is crucial for providing support for other languages. An implementation of Welkin will contain a small section of ASCII encoding for easier standardization, but the rest of the program can be done *entirely* in the user's native language. This is a novel feature in most programming languages, which are either predominantly English or are fine tuned for specific human languages.

2.6. **Bootstrapping.**

There are two important considerations in Welkin: how the base operations are implemented, and how proofs are expressed. In previous revisions, the author separated these aspects, particularly when describing the syntax and semantics of the language. However, this left many proofs in English, making it difficult to transfer.

[TODO: provide citations] The author's emphasis of proofs stems from formal methods, the study of providing mathematical proof to verify computer programs. A central notion in this field is the **Trusted Computing Base (TCB)** of a program, which is code used to implement it, as well as the axioms used to prove its correctness. Many authors advocate for a small, fixed TCB. However, this is inefficient in practice… Setting for a single system, like ZFC, does not resolve whether Welkin is universal *for proofs*.

To resolve both issues, we "bootstrap" Welkin from the ground up.

- First, we show that Welkin can express any computer program, as well as a verifier for its own claims.
- Next, we define the syntax using the notion of **invertible syntax descriptions**, introduced in [RO10]. This will allow us to define the syntax *entirely* in Welkin. We will also prove the syntax is **unambiguous**, or that given any string, if it is accepted, there must be one way to process it in the program.

[5] Contexts can mimic regular comments, but Welkin aims to be inspectable by users. Encapsulation or private information can be enforced through *rejecting* specific contexts or only accepting certain ones.

DRAFT

- Then, in Section 5, we prove that our definition of information is complete with respect to being accepted by a Turing machine.

3. Foundations

This section discuss the foundations of Welkin, as follows:

1) We define words and handles.
2) We define units and their rules.
3) We prove that is a correspondence between units and Turing machines.
4) We define queries.

For this thesis, definitions and proofs will be given at a high level. Additionally, we adopt several conventions:

- Each **Definition**, **Remark**, and **Example** ends with a triangle ($\triangle$).
- Each proof ends with a square ($\square$).
- We frequently abbreviate "if and only if" as "iff".

3.1. Words and Handles.

For high-level notation, we write $a \equiv b$ to mean that a is definitionally equivalent to b. High level notation will *not* be stated in the syntax unless where noted. We will be pedantic in most notation for complete precision. Moreover, implementation details are outside the scope of this thesis. This includes, e.g., practically restricting user input by fixed upper bounds.

Definition 3.1.0. A **bit** is the symbols 0 or 1. A **binary word** is either the symbol ε (the **empty word**), and if w is a finite binary word, so are $w0$ and $w1$, where the juxtaposition of symbols denote **concatenation**. Nothing else is a word. $\triangle$

We also require a notion of equality on bits. To ensure this is constructive, we provide a *separate* definition of inequality as well. The latter provides an explicit certificate, a bit that shows how two words are distinct.

Definition 3.1.1. Equality $=_{\mathrm{W}}$ and inequality $\neq_{\mathrm{W}}$ on words w_1, w_2 is defined recursively and as nothing else:

- **Base case:** $\varepsilon = \varepsilon$.
- **Recursive step:** suppose $w_1 =_{\mathrm{W}} b_1 w_{1'}$ and $w_2 =_{\mathrm{W}} b_2 w_{2'}$, where b_1, b_2 as bits and $w_{1'}, w_{2'}$ are words. Then $w_1 =_{\mathrm{W}} w_2$ if and only if b_1, b_2 are both 0 or 1, and $w_1' =_{\mathrm{W}} w_2'$. Moreover, $w_1 \neq_{\mathrm{W}} w_2$ if and only if b_1 is 0 and b_2 is 1, b_1 is 1 and b_2 is 0, or $w_1' \neq_{\mathrm{W}} w_2'$.

$\triangle$

Words alone do not carry meaning. Instead, meaning is provided through **handles**.

Definition 3.1.2. A **handle** is given by an **ID**, which is a word. The interpretation of a given handle is a free parameter, and therefore outside the scope of this language. $\triangle$

Because handles act as free parameters, we work with them through truth. Based on this, the rest of the thesis will abstract away general units *as* sets of axioms. Certain interpretations are used to define Welkin's syntax. For example, words can be interpreted as handles without external meaning; see Section 4.

DRAFT

Now, we can define equality and inequality between handles. We will reserve $=$ and $\neq$ for handles, respectively.

Definition 3.1.3. Consider two handles h_1, h_2, and let ID_1 and ID_2 be their respective IDs. Then:

- h_1 is **equal** to h_2, written $h_1 = h_2$, if and only if $\mathrm{ID}_1 =_{\mathrm{W}} \mathrm{ID}_2$.
- h_1 is **not equal** to h_2, written $h_1 \neq h_2$, if and only if $\mathrm{ID}_1 \neq_{\mathrm{W}} \mathrm{ID}_2$.

$\triangle$

3.2. **Units.**

Now we can define units. Roughly, units are finite combinations of handles and representations. We present the complete definition as follows.

Definition 3.2.4. A **unit** is defined recursively as follows and nothing else:

- **Base case:**
  - The symbol $\{\}$, the **empty block**.
  - A handle, see Definition 3.1.9.
- **Recursive step:** let u, v, g be units and h a handle. Then any finite combination of the following are also units:
  - A **pair** $\{u, v\}$.
  - A **representation** $a - c \to b$, where a is the **sign**, c is the **context**, and b is the **referent**. This is read as: a **represents** b **in context** c.

$\triangle$

We add several abbreviations, most of which will appear in Section 4:

- $\{a\}$ denotes $\{a, \{\}\}$.
- $a \leftarrow c \to b$ denotes that both $a - c \to b$ and $a \leftarrow c - b$ hold.
- We add a symbol $\sqsubseteq$ for **containment**, where $a \sqsubseteq g$ iff $a - g \to a$. When writing *in* the language, we will prefer the latter form.

Now we may introduce the rules on units.

[TODO: fix wrapping of long table! Want to make this clearer!]

DRAFT

Definition 3.2.5. All rules in Table 3 hold, stated over meta-variables a, b, c, d, g for units and h_1, h_2 for handles.

Rule	Name	Content
R1	**Internal Transitivity**	$a - c \to b$ and $b - c \to d$ imply $a - c \to d$
R2	**Pair Congruence**	If $a - c \to b$, then $\{a, d\} - c \to \{b, d\}$
R3	**Sign Congruence**	If $a - c \to b$, then $\{a - g \to d\} - c \to \{b - g \to d\}$
R4	**Context Congruence**	If $a - c \to b$, then $\{d - a \to g\} - c \to \{d - b \to g\}$
R5	**Referent Congruence**	If $a - c \to b$, then $\{d - g \to a\} - c \to \{d - g \to b\}$
R6	**Contextual Lifting**	$a - c \to b$ and $d - a \to g$ imply $\{d - b \to g\} \sqsubseteq c$
R7	**Refinement**	$\{a - c \to b, a - c \to d\} \longrightarrow \{a - c \to b\}$
R8	**Empty**	$\{g, \{\}\} \leftarrow c \to g$
R9	**Sign Absorption**	$\{\{\} - a \to b\} - c \to \{\}$
R10	**Context Absorption**	$\{a - \{\} \to b\} - c \to \{\}$
R11	**Referent Absorption**	$\{a - b \to \{\}\} - c \to \{\}$
R12	**Handle Equality**	If $h_1 = h_2$, then $h_1 \leftarrow c \to h_2$
R13	**Containment**	$\{a - g \to a\} \leftarrow c \to \{\{g, a\} \longleftrightarrow g\}$
R14	**Idempotentcy**	$\{a, a\} \leftarrow c \to a$
R15	**Associativity**	$\{a, \{b, c\}\} \leftarrow c \to \{\{a, b\}, c\}$
R16	**Commutativity**	$\{a, b\} \leftarrow c \to \{b, a\}$

TABLE 3. Set of all valid rules in Welkin.

△

We review the utility of each rule. Note **R6** is the only rule *between* contexts; the rest is entirely user defined. Moreover, only **R1**, **R3**, and **R4** are needed for Turing completeness. However, the other rules are in place to help with organizing units as modules, as well as make it easier to use the language.

[TODO: refer to rules with ranges when appropriate! Much nicer.]

- **R1**, **R2**, **R3**, **R4**, **R5**, **R6** were discussed in Section 2.2.[TODO: maybe review the discussion from earlier? Might be useful to reinforce main ideas to reader.]
- **R7** is explained as follows: suppose a is a unit, and in a block, a represents two other units b, d. Then this block *represents* $\{a \longrightarrow b\}$ and $\{a \longrightarrow d\}$, separately. This provides a mechanism to *refine* a general unit into a more specific one.
- **R8**, **R9**, **R10**, and **R11** define the behavior of the empty unit $\{\}$, similar to the empty set. **R9** and the like specifically states that $\{\}$ contains *no* representations. Thus, if a representation has a component that is $\{\}$, then it carries no meaning.

DRAFT

This provides a mechanism to *exclude* units in a context. This is useful for imposing invariants.
- **R12** enables equality in words and handles to pass through into representations. Besides this, note that equivalences on units are entirely user defined.
- **R14**, **R15**, and **R16** ensure that information can be repeated and arranged in any order. Mathematically, this means that units have a **semi-lattice** structure.

Remark 3.2.6. The rules in Definition 3.2.12 use few meta-variables, which may *or* may not equal each other. This is connected to variables as managed by the proof checker MetaMath Zero [Car20]. In first-order logic, variables in quantifiers are assumed to be distinct. To *allow* for equality as well, these must be separately included, but this quickly explodes in size. All of this means that Welkin, similar to MetaMath Zero, achieves a significant level of compression. However, *direct* conversions to first-order theories may not be feasible. △

As more notation, we write:
- $\sim a$ for $\{a \longrightarrow \{\}\}$.
- $*\{a_1, ..., a_n\} - c \to b$ for $\{a_1 - c \to b, a_2 - c \to b, .., a_n - c \to b\}$.
- $a - c \to *\{b_1, ..., b_n\}$ for $\{a \longrightarrow b_1, a \longrightarrow b_2, .., a \longrightarrow b_n\}$.

We will officially add $\sim$ and $*$ to the grammar in Section 4.

3.3. **Turing Completeness.**

This section shows that Welkin is Turing complete. For background, there are many papers, e.g., [Sip13].

We want to demonstrate that every unit corresponds to some Turing machine, and vice versa. More precisely, we want to find a one-to-one mapping φ from units c to a Turing machine φ_c such that

$$a - c \to b \text{ if and only if } \varphi_c(\langle \varphi_a \rangle) = \langle \varphi_b \rangle. \qquad (\star)$$

Here, $\langle \varphi_a \rangle$ denotes a standard encoding of a Turing machine as a string (see [Sip13]). We also require that φ is surjective, i.e., each Turing machine T can be expressed as φ_u for some unit u. The purpose of φ is to provide a computational interpretation of units *as* programs, or operators. Note that handles can have implicit, user-defined meaning (see Definition 3.1.9). Practically, we will maintain this interpretation, as representations are managed mechanically through truth[6]

At this point, we are concerned with showing *some* φ exists, thereby validating Goal 1 [TODO: provide a link to this]. We are not concerned about efficiency; this will be left for a future work, see Section 6. To construct φ, we want to create an embedding into the SK-combinator calculus. This is an equational theory, first developed by Schönfinkel [Sch24], and independently discovered by Curry [Cur30]. In this theory, a **combinator** is a higher-order function: a function that takes in other functions as inputs.[7] As a simplification, we present this calculus using a **reduction relation** instead of equality.

[6]In connection to linguistics, this is the difference between a formal semantics, what is stated in the language, and pragmatics, the intension or purpose of a term.

[7]A remark for logicians: this calculus is extremely similar to a Hilbert-style proof system, with K and S corresponding to the rules $(\varphi \Rightarrow (\psi \Rightarrow \varphi))$ and $(\varphi \Rightarrow (\psi \Rightarrow \zeta)) \Rightarrow ((\varphi \Rightarrow \zeta) \Rightarrow (\psi \Rightarrow \zeta))$, respectively. This was one of Curry's insights in connecting logic to computation [Cur30].

DRAFT

Definition 3.3.7. The SK-combinator calculus consists of the following:

- A **term** is defined recursively as either K or S, and if M, N are terms, so is (M) and their **application** MN.
- Evaluation: M **reduces** to N, written $M \Rightarrow_{\text{SK}} N$, if their equality can be deduced from the following axioms.
  - **Base Axioms:** for all terms A, B, P:
    - $((KA)B) \Rightarrow_{\text{SK}} M$
    - $(((SA)B)P) \Rightarrow_{\text{SK}} (AP)(BP)$.
  - **Transitivity:** if $M_1 \Rightarrow_{\text{SK}} M_2$ and $M_2 \Rightarrow_{\text{SK}} M_3$, then $M_1 \Rightarrow_{\text{SK}} M_3$.
  - **Congruence:** if $M_1 \Rightarrow_{\text{SK}} M_2$ and $N_1 \Rightarrow_{\text{SK}} N_2$, then $M_1N_1 \Rightarrow_{\text{SK}} M_2N_2$.

△

Now, Welkin can be embedded into this calculus. We discuss this embedding at a high level:

- Each unit Definition 3.2.11 can be built from handles, or by finite combinations of pairs and representations. These can be represented as combinators through suitable encodings of positive integers and finite tuples, respectively. For more details, refer to [Tur79].
- Each rule in Definition 3.2.12 are themselves finite, ranging over a finite number of meta-variables. These, too, can be encoded as combinators through if/else statements.

It remains to show that every combinator is included in this embedding. For this, we prove the following. Our proof technique uses recursion within Welkin.

Theorem 3.3.8. *Let T be a term in the* SK*-combinator calculus. Then there is some unit u such that $T = \varphi_u$.*

Proof. We prove there is a unit L that can generate a unit u_T for each term of the SK calculus. To this end, we use K and S to represent the corresponding combinators in Definition 3.3.13, as well as M, N, P to represent meta-variables. With these, L is defined with exactly the following:

- $L - L \to *\{K, S\}$: recall that this is equivalent to $L - L \to K$ and $L - L \to S$. These units represent that K and S, respectively, are terms.
- $*\{M, N, P\} - L \to L$: recall that this is equivalent to $M - L \to L$, $N - L \to L$, and $P - L \to L$. Each of these represent meta-variables through **R1**. For example, $M - L \to K$ holds, but also $M - L \to S$. In other words, M can *potentially hold* K or S.
- $L - \{N - M \to L\}$: composition MN of two terms M, N is represented as $\{N - M \to L\}$, which itself is a term.
- $\{N - \{M - K \to L\} \to L\} - L \to M$: represents the reduction rule for K.
- $\{P - \{N - \{M - S \to L\} \to L\} \to L\} - L \to \{N - \{M - K \to L\} \to L\}$: represents the reduction rule for S.

We claim that L represents $\Rightarrow_{\text{SK}}$. First, to prove closure under composition, suppose $L - L \to A$ and $L - L \to B$. Then by **R1**, $M - L \to A$ and $N - L \to B$, so by **R3** and **R4**, $\{N - M \to L\} - L \to \{B - A \to L\}$. Another application of **R1** yields $L \longrightarrow \{B - A \to L\}$. The rules $L \longrightarrow K \mid S$ and $L - L \to \{N - M \to L\}$ are the only terms with sign L (besides L itself), so L contains *exactly* the terms in the SK-calculus.

DRAFT

Second, the base axioms for K and S are already included, and transitivity is provided by **R1**.

Finally, **R4** entails term congruence: if $M - L \to M'$, then $N - M \to L$ represents $N - M' \to L$. Because $\underset{\text{SK}}{\Rightarrow}$ and any term can be represented in L, this completes the proof. □

Taken together, we have completed *one* part of Goal 1 [LINK]. The other part will be addressed in the next section.

3.4. **Queries and Information.**

By Theorem 3.3.1, every partial computable can be expressed as a unit. Additionally, in the construction used, reductions of terms are *also* represented. This provides a ceiling on what queries we can *express* computably. For more details, refer to [HMU07]. This asks whether a representation is contained in a context.

Definition 3.4.9. Let c be a context. Then a **query over c** is a question over fixed units p, q, d: "Does $\{p - q \to d\} - \text{unit} \to c$ hold?" △

We can then apply unit in an attempt to evaluate this. In some cases, this is easy and only uses *direct definitions*. Consider, for example, examining p in context $\{p, q\}$. We can resolve this with a quick traversal, showing that p is in fact in $\{p, q\}$. However, Theorem 3.3.1 implies that finding general derivations are uncomputable. More precisely, we have the following.

Corollary 3.4.10. *The problem of determining whether a query is satisfiable is RE-complete.*

We want information to *resolve* a query. Formally, we want information to be *proof*, not a "yes/no" answer by itself. When the context is finite, like $\{p, q\}$, there is a direct proof. But what about the general case? Some queries may reduce *forever*. While we cannot determine *which* ones reduce to $\{\}$ or not, we *can* find the ceiling of proofs expressible by computable functions. This process is more involved and is the subject of the next section.

[TODO[MEDIUM]: make this definition self-contained! The goal is to show that this *is* complete. That should be independent of the next section!]

Definition 3.4.11. Information about a query q in context c is a any partial meta-proof in meta of a derivation of $\{c, q\}$. △

4. Syntax

This section examines the grammar for Welkin and provides validation rules. For specific sections, consult Table 4.

DRAFT

Section Number	Title	Description
Section 4.1	**Encoding**	Introduces the encoding for Welkin.
Section 4.2	**Invertible Syntax Descriptions**	Defines **invertible syntax descriptions**, the fundamental building blocks for the grammar.
Section 4.3	**Strings and IDs**	Defines the syntax for strings and IDs.
Section 4.4	**The Welkin Grammar**	Presentts the Welkin grammar, defining which strings are syntactically allowed.
Section 4.6	**Proof of Unambiguity**	Proves that the Welkin grammar is unambiguous.
Section 4.5	**Validation and Transforms**	Defines which strings are **valid**, as well as transforms on certain constructs.

TABLE 4. Overview of Section 4.

DRAFT

To make specific characters clearer, we write syntactic features in `type-writer font`.[8] For clarity, we will write at a slightly higher level than Section 3.

4.1. **Encoding.**

Welkin uses ASCII as its base encoding. The term ASCII is slightly ambiguous, as there are subtly distinct variants, so we formally define US-ASCII as a standard version.[9]

[8]This font is Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1 (https://github.com/intel/intel-one-mono/blob/main/OFL.txt).

[9]Note that this table *itself* is a representation, which represents glyphs with binary words.

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
32	20	Space	56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m
38	26	&	62	3E	>	86	56	V	110	6E	n
39	27	'	63	3F	?	87	57	W	111	6F	o
40	28	(	64	40	@	88	58	X	112	70	p
41	29	)	65	41	A	89	59	Y	113	71	q
42	2A	*	66	42	B	90	5A	Z	114	72	r
43	2B	+	67	43	C	91	5B	[	115	73	s
44	2C	,	68	44	D	92	5C	\	116	74	t
45	2D	-	69	45	E	93	5D	]	117	75	u
46	2E	.	70	46	F	94	5E	^	118	76	v
47	2F	/	71	47	G	95	5F	_	119	77	w
48	30	0	72	48	H	96	60	`	120	78	x
49	31	1	73	49	I	97	61	a	121	79	y
50	32	2	74	4A	J	98	62	b	122	7A	z
51	33	3	75	4B	K	99	63	c	123	7B	{
52	34	4	76	4C	L	100	64	d	124	7C	|
53	35	5	77	4D	M	101	65	e	125	7D	}
54	36	6	78	4E	N	102	66	f	126	7E	~
55	37	7	79	4F	O	103	67	g			

TABLE 5. Printable US-ASCII codes and glyphs.

DRAFT

Definition 4.1.0. Welkin's encoding consists of **Printable US-ASCII**, listed in Table 5, as well as character `EOF`, with code 58. An **ASCII string** is the concatenation of any finite number of printable characters, followed by an `EOF` character. △

To represent general encodings, there is a binary format supported for strings, see Listing 1.

Moreover, we will need to define **character classes**, or important sets of characters. These are presented in Table 6, with each character separated by |. Note that strings *may* contain quotes prefixed with a slash \, see Listing 1.

Set Name	Characters
PRINTABLE	See Table 5.
WHITESPACE	\t | \r | Space
DELIMITER	{ | } | ' | " | . | ,
RESERVED	DELIMITER | * | @
SQ_CHAR	Any PRINTABLE character except ' and \
DQ_CHAR	Any PRINTABLE character except " and \
ID_CHAR	Any PRINTABLE character not in RESERVED or WHITESPACE

TABLE 6. Definitions of key character classes. Each class is denoted in UPPERCASE.

4.2. Invertible Syntax Descriptions.

To more easily describe the language, we provide basic building blocks. For simplicity, we do not add many primitives. The names are taken from the the `parsec` combinator library in the programming language Haskell [LM01]. Using [RO10], we augment these functions as invertible syntax descriptions. Here, an **invertible syntax description** is a pair of functions:

- A `parser`, from strings to an intermediate data structure.[10]
- A `printer`, the reverse of `parser`.

The detailed correspondence is described in Section 4.2. We adapt the notation from `parsec` and [RO10] for our purposes:

[TODO: define how printing works here! Use a table.]

- `seq` means two rules that must be concatenated *directly*, no `WHITESPACE` characters allowed.
- `lexeme` means two rules can be joined with any number of `WHITESPACE` characters in between.
- `seq_many_till` means that in `A - many_till -> B`, parse zero or more instances of `A` *without* any `WHITESPACE` characters, until `B` is encountered.
- `lex_many_till` means that in `A - many_till -> B`, parse zero or more instances of `A` *with* any number of `WHITESPACE` characters, until `B` is encountered.

In contrast to [LM01], however, we include the ability *print* as well or present the corresponding string. This is done through [RO10].

4.3. Strings and IDs.

Strings allow escaped single or double quotes, see Listing 1. Note that, semantically, single and double quoted strings are equivalent, see Section 4.5.

```
STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR, "\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR, '\"'} - seq_many_till -> '"',
```

LISTING 1. Strings.

[10] For computer scientists: this is an AST. Because this thesis aims to have a broader audience, we leave details to [RO10].

DRAFT

IDs are special cases of strings that do not require quotes but forbid whitespace and certain characters, see Listing 2.

```
ID := ID_CHAR - seq_many_till -> *{RESERVED, *WHITESPACE}
```

LISTING 2. Syntactic definition of an ID.

4.4. **The Welkin Grammar.**

Welkin's grammar is displayed in Listing 3, inspired by several languages.

DRAFT

```
start := units - lexeme -> EOF,

units :=
  | ""
  | {
      unit
      - lexeme -> {"," - lexeme -> unit}
      - lex_many_until -> *{"", ","}
    },

unit := node - lexeme -> *{"", arc - lexeme -> unit},

arc := right_arc | other_arc,
right_arc := "-" - seq -> node - seq -> "->",
other_arc := "<-" - seq -> node - seq -> *{"-" | "->"},

node := "*" - seq_many_till -> *{
    graph,
    path - lexeme -> *{"", binding}
  }

binding := ":=" - lexeme -> choices,
choices := *{"", "|"}
  - lexeme -> unit
  - lexeme -> *{
    "",
    {"|" - lexeme -> unit}
    - lex_many_til -> *{"}", ","}
  },

graph := path - lexeme -> "{" - lexeme -> units - lexeme -> "}",

path := *{"", "~"}
  - seq -> *{"", "@"}
  - seq -> HANDLE
  - seq -> trailer,

trailer - *{
  "",
  "." - seq -> *{graph, HANDLE - seq -> trailer}
}

HANDLE := ID | STRING

STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR, "\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR, '\"'} - seq_many_till -> '"',

ID := ID_CHAR - seq_many_till -> *{RESERVED, WHITESPACE}
```

LISTING 3. The complete grammar for Welkin. This includes the rules STRING and ID, which are also provided in Listing 1 and Listing 2, respectively. Character classes are defined in Table 6.

4.5. **Validation and Transformations.**

We say a string is **valid** if it is accepted by the grammar (Listing 3), and the following hold:

DRAFT

- The number of dots (relative import) used must not exceed the number of levels available.
- There must be at least one path that starts with `@h`, where `h` is some handle. This is used to name the module.
- For all paths that start with `@h1` and `@h2`, respectively, `h1` must be equal to `h2`.
- Every definition of a unit must only be stated once. This means `u := v, u:= w` is not allowed.
- Accessing a unit v from u requires that u has a closed definition containing v. This means neither `u := {v, w}, u.x` nor `u.x` alone are valid.

We leave error handling for future work, see Section 6.

Additionally, we define transformation rules after parsing:

- Some node that starts with `@` is found. This marks the module name. Denote this name as m.
- If `\'` appears in a single quoted string, then this will be replaced by `'` in the final contents. For example, given a string `"John\'s dog"`, the contents would be `John's dog`.
- A similar rule applies for `"` but with `\"`.
- Double slashes `\\` are converted into one slash `\`.
- Single and double quoted strings are represent each other. For example, `'hello' <--> "hello"`. In fact, IDs are special cases of strings, so `hello <--> 'hello'`. Practically, the only difference is that units forbid `WHITESPACE` or `RESERVED` characters, but they can be written without quotes.
- Paths are resolved, determined where the unit originated. For example, in `a, c := {.a, b}`, the `.a` inside `c` would be expanded as `a`.
- Each `*u` for a handle or graph `u` expands to all the members in the enclosed unit. If `u` contains a at most one item or is a representation, then `*u` is expanded as `u`. Multiple `*`, such as `**u`, recursively expands the contents in a unit.
- Each definition `x := a1 | ... | an` expands to `x <--> {x - x -> a1, ..., b - x -> an}`. For each graph `a1`, the generated graph will contain `v - x -> v`, where `v` is a handle or representation in `a1`.
- Each `~x` expands to `x - g -> {}`, where `g` is the name where `~x` is placed. At the highest level, this would be the module name, m.
- The definitions for `|` and `~` are expanded, see Definition 3.2.12.
  - –
  - –

4.6. **Proof of Unambiguity.**

We show that, by construction, the combinators we used form an $\mathrm{LL}(1)$ grammar. These grammars have two desirable properties:

- If a string is accepted, it is parsed unambiguously.
- Efficient parsers can be easily and efficiently implemented [Aho+06].

Our approach is to provide an equivalence between Listing 3 and a new grammar. More precisely, we require a bijection with the following property: a string accepted by Listing 3 is also accepted by the new grammar, and vice versa. We will then prove the latter is $\mathrm{LL}(1)$.

We will keep definitions and theorems here self-contained. For more background, please consult [Aho+06], [RS70].

DRAFT

First, we need to define a general **context-free grammar**.

Definition 4.6.1. A **Context-free Grammar (CFG)** $G = (N, T, P)$ consists of:

- A finite set of **non-terminals** N.
- A finite set of **terminals** T.
- A finite set of **productions** P, where a **production** is a rule of the form $A \Rightarrow \alpha_1 ... \alpha_n$. Here, A is a non-terminal, $\alpha_1 ... \alpha_n$ denotes concatenation, and each α_i may be a non-terminal or terminal. Note that this string may be empty, denoted by EPS.
- A **start symbol** $S \in N$.

△

For our use case, we will assume that T is a set of ASCII strings (Definition 4.1.16). Additionally, we define a **string of grammar symbols** as a string of the form $\alpha_1 ... \alpha_n$, with each α_i being a terminal or non-terminal. Additionally, we will abbreviate `WHITESPACE` in a CFG as WS, and $A \Rightarrow \alpha \mid \beta$ means the grammar includes the productions $A \Rightarrow \alpha$ and $A \Rightarrow \beta$.

We require another important definition.

Definition 4.6.2. Let $G = (N, T, P, S)$ be a CFG. Let $A \in N$ and $\alpha, \beta, \gamma \in T$. Then $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if there is a production $A \Rightarrow \gamma$ in P. This is called a **one-step derivation**. More generally, α **derives** β, denoted $\alpha \overset{*}{\Rightarrow} \beta$, if there is some finite sequence $\alpha_1 \equiv \alpha \Rightarrow \alpha_2 \Rightarrow ... \Rightarrow \alpha_n \equiv \beta$. The **language** of G, denoted $L(G)$, is the set of all strings s such that $S \overset{*}{\Rightarrow} s$. △

Next, we need to convert the combinators in Section 4.2 into CFG productions. These are based on [CITE], which proves that the combinators accept exactly the same set of strings as their transformed productions.

- `A - seq -> B` corresponds to A_B ⇒ A B.
- `A - lexeme -> B` corresponds to A_B ⇒ A WS_star B, where WS_star ⇒ WS WS_star | ε.
- `A - seq_many_till -> B` corresponds to A_star_B ⇒ A_star B, where A_star ⇒ A A_star | ε.
- `A - lex_many_till -> B`corresponds to A_lex_star_B ⇒ A_lex_star WS_star B where A_lex_star ⇒ A WS_star A_star | ε.

$$\text{start} \Rightarrow \text{unit WS} \tag{1}$$

FIGURE 1. CFG for the Welkin grammar, based on Listing 3.

Substituting these produces Figure 1, and we obtain the following theorem.

Theorem 4.6.3. *The Welkin grammar (Listing 3) accepts exactly the same ASCII words as Figure 1.*

DRAFT

Definition 4.6.4. ([Aho+06]). Let $G = (N, T, P)$ be a CFG and string of grammar symbols A.

- The **FIRST** set, denoted $\mathrm{FIRST}(A)$, consists of all β such that $\alpha \Rightarrow \beta\gamma_1 ... \gamma_n$, as well as ε if the grammar contains a a production $\alpha \Rightarrow \varepsilon$.
- The **FOLLOW** set, denoted $\mathrm{FOLLOW}(A)$, is the set of all non-terminals A such that, for some α and β, $S \overset{*}{\Rightarrow} \alpha A a \beta$.

△

Using this definition, we are now ready to define $\mathrm{LL}(1)$ grammars.

Definition 4.6.5. ([Aho+06]). A CFG is $\mathrm{LL}(1)$ if and only if for every production $A \Rightarrow \alpha \mid \beta$:

- At most one of α or β can derive ε.
- $\mathrm{FIRST}(\alpha)$ and $\mathrm{FIRST}(\beta)$ are disjoint.
- if $\varepsilon \in \mathrm{FIRST}(\beta)$, then $\mathrm{FIRST}(\alpha)$ and $\mathrm{FOLLOW}(A)$ are disjoint. A similar condition is required on β if $\varepsilon \in \mathrm{FIRST}(\alpha)$.

△

Now, we work on the proof that the new grammar is $\mathrm{LL}(1)$.

Theorem 4.6.6. *The grammar in Figure 1 is* $\mathrm{LL}(1)$. *Hence, the Welkin grammar Listing 3 is unambiguous.*

Proof. We first observe that the character classes in Table 6 are disjoint from each other.

- Strings are enclosed in quoets and do not overlap with RESERVED or WHITESPACE.
- IDs are defined to be disjoint from RESERVED or WHITESPACE.

It remains to show that all other applicable rules satisfy Definition 4.6.20. For this, see the calculations in Table 7. Because the intersections in this table are all empty, and because at most one derivation derives ε in a given choice, this proves that Figure 1 is $\mathrm{LL}(1)$. Thus, by Theorem 4.6.3, the Welkin grammar accepts the same strings as an unambiguous language.

Rule	Sets	Set One	Set Two	Intersection
unit				

TABLE 7. Calculations needed for $\mathrm{LL}(1)$ proof.

□

By Theorem 4.6.3, we immediately obtain the following.

Theorem 4.6.7. *The Welkin grammar (Listing 3) is unambiguous.*

5. METATHEORY

[TODO: need a quick way to *establish* the main definitions for information, then *justify* here why they are complete.]

DRAFT

This section discusses the provably most general definition of information. This section is optional. For the base definition of information, refer to Definition 3.4.15.

[TODO: convert this into a table.]

1) We show that Welkin's base theory is equivalent to a weak fragment of arithmetic, $I\Delta_0$.
2) We overview Artemov's work in serial consistency. [Art24].
3) We establish the meta-theory, and show it can encompass any formal system.
4) We show the meta-theory "meta-proves" its own soundness, *without* contradicting known impossibility results.
5) We conclude with the definition of information.

[TODO: define language of arithmetic!]

We will make two assumptions without loss of generality:

- First, we will consider only first-order theories T that are RE. This is because any formal system must be defined with proofs accepted by some Turing machine. In turn, this Turing machine can be expressed as a first order theory, see [Tra50]. [TODO: maybe make more clearer claim?]
- Secondly, we will only consider theories in the language of arithmetic [TODO: maybe define?]. A first-order theory can be encoded via arithmetic, similar to the assumption above.

DRAFT

5.1. Establishing the Floor: $I\Delta_0$.

To compare Welkin against other theories, we show the unit verify can be translated to $I\Delta_0$, a weak fragment of arithmetic, and vice versa. This subsection is optional, and we will keep the proofs at a high-level for readability. For background in first-order logic, please refer to [Men09].

Herein, let $a \Leftrightarrow b$ denote a if and only if b. Robinson Arithmetic denotes the base set of axioms; refer to [HP17], which use $I\Sigma_0$ to denote $I\Delta_0$.

[TODO[SMALL]: fix equation labels!]

Definition 5.1.0. Robinson Arithmetic Q is the first-order theory over the language of arithmetic with the following axioms, universally quantified over x, y, z:

- **Q1:** $\neg(S(x) \neq 0)$.
- **Q2:** $S(x) = S(y) \Rightarrow x = y$.
- **Q3:** $x \neq 0 \Rightarrow \exists y.x = S(y)$.
- **Q4:** $x + 0 = x$.
- **Q5:** $x + S(y) = S(x + y)$.
- **Q6:** $x * 0 = 0$.
- **Q7:** $x * S(y) = (x * y) + x$.
- **Q8:** $x \leq y \equiv \exists z.z + x = y$

$\triangle$

Definition 5.1.1. The theory $I\Delta_0$ [PW81] consists of Q plus the **bounded induction schema**:

$$(\varphi(0) \wedge \forall x.(\varphi(x) \Rightarrow \varphi(x+1))) \Rightarrow \forall x.\varphi(x)$$

for each φ with bounded quantifiers, which means quantifiers $\exists x < t.\psi(x,t)$ and $\forall x < t.\psi(x,t)$ where x is free in term t and $\psi(x,t)$ is quantifier free. △

Remark 5.1.2. Note that the induction schema is stronger than having open formulas. This allows statements about, e.g., odd and even numbers to be proved. We will need this to express $\mathrm{verifier}$. △

[TODO[MEDIUM]: make this more rigorous.]

Lemma 5.1.3. *The unit* $\mathrm{verifier}$ *is definable in* $I\Delta_0$.

Proof Sketch. The claim relies on defining unit. From there, one can easily express the conditions in $\mathrm{verifier}$ by simple recursion.

To this end, we first argue that the inductive definitions can be written in $I\Delta_0$. Clearly, every handle can be expressed, indexing each triple of functions with Cantor's pairing function, sending triples $(\mathrm{UID}, \mathrm{RID}, \mathrm{HID})$ to natural numbers. Similarly, representations can be indexed by a pairing argument. It remains to show that blocks can be defined as well. We claim that an extended pairing function can be made that is defined inductively. [TODO: define this function!]

Second, it can be easily shown that each rule in Definition 3.2.12 are definable by induction, in at most 7 variables. □

An important consequence of this theorem is the following, proving that the metatheory for Welkin is as minimal as possible.

Now we proceed that Welkin's $\mathrm{verifier}$ is itself can process any $I\Delta_0$ proof.

Lemma 5.1.4. *There is a unit* Q *in Welkin and a bijection from propositions* φ *units* u_φ *such that* $Q \vdash \varphi$ *if and only if* $u_\varphi - \mathrm{part} \to \mathrm{Q}$.

Proof. Let Q be a new handle. Encode the naturals into words via Definition 3.1.7 and Definition 3.1.8, with successor represented through pairs. [TODO: define addition and multiplication via units, in the direct way.]

Axioms **Q1**-**Q3** easily follow from []. The remaining axioms are included in Q. [TODO: note how quantifiers are correctly expressed in a separate lemma]. Using transitivity and ?, this completes the proof. □

Theorem 5.1.5. *Welkin's* $\mathrm{verifier}$ *verifies there is a context with all derivations of* $I\Delta_0$.

Proof. We prove that Q, constructed in Lemma 5.1.7, combined with the $\mathrm{verifier}$ unit proves the claim. TBD. □

Taken together, we can prove that Welkin has a minimal metatheory.

Corollary 5.1.6. (Base Theory: $I\Delta_0$). *Suppose* T *is another first order theory that proves the existence of Welkin's* $\mathrm{verifier}$. *Then* $T \Rightarrow I\Delta_0$.

Proof. Over $I\Delta_0$, Definition 5.1.22 proves the existence of $\mathrm{verifier}$ implies $I\Delta_0$. Thus, if T proves the existence of $\mathrm{verifier}$, it must satisfy $I\Delta_0$ as well, completing the proof. □

DRAFT

5.2. **Artemov's Approach.**

A major goal in Welkin is to express any representable notion, *including* any representabale proof. Proofs are a finite certificate that consists of a valid sequence of steps. Importantly, proofs can be verified using a *total* computable function, one that can check *any* given proof. Because of the requirement for computable checking, limitations in computable functions apply. These are illustrated by several impossibility theorems in the literature, including Gödel's infamous incompleteness theorems [Göd06].

However, previous impossibility results rely on proving strong properties of proofs. Artemov argues that weaker, constructive properties can be used instead. We provide a high-level definition of **serial-consistency** before proceeding onto the embedding in Welkin.

Definition 5.2.7. Suppose T is sufficiently strong to express its own proofs. Then T is **serial-consistent** if there is a **selector** for T. A **selector** is a computable function s [11] with two properties:

- s accepts any string that encodes a valid proof.
- if s accepts a proof, then it cannot contain a contradiction.

△

Artemov overcomes Gödel's second incompleteness theorem. This theorem states that Peano Arithmetic PA, providing basic properties of the natural numbers, cannot prove its own consistency. Artemov instead demonstrates that PA proves its own *serial-consistency*, as follows:

- He uses a weak meta-theory, in this case, Primitive Recursive Arithmetic (PRA). This theory is bounded by a weak principle of induction.
- Artemov encodes all proofs as natural numbers in PRA. PRA is sufficiently strong *just* to encode all of these proofs.
- Finally, he uses a proof by induction, over encoded proofs in PRA, to show that PA proves each *individual* proof contains no contradictions. This is possible through partial truth definitions.

To be clear: Artemov does *not* show that there is a *single* proof that PA is consistent, that works for all proofs. Instead, his method *takes* a proof as a parameter. There have been extensive discussion on the validity of this technique, and its acceptance by other logicians.[12]

Despite its initial controversy, Artemov's techniques follow closely with several constructive schools, see [Art24]. This thesis builds upon this result with a stronger property: **serial-soundness**. The exact same proof applies, using the same partial truth definition. Tarski's theorem says that a formal system cannot define its own truth predicate *at the object language* [Tar83]. In fact, this truth predicate is uncomputable, in general. We do not avoid Tarski's theorem, but we do approximate it *as* closely as partial computable functions allow. Because some logics may not have a notion

[11] Technically, this is a primitive recursive function, but we will quickly generalize this to *any* computable function.

[12] The discussion is available online at: https://mathoverflow.net/questions/469247/situation-with-artemovs-paper.

DRAFT

of inconsistency, we will not define consistency itself. However, given the results in Section 3, first order logic can be expressed in several ways.

5.3. **Truth and Soundness.**

We embed "Convention T", or Tarski's criterion for truth [Tar83], as unit. Recall that, at this stage, Welkin deals with truh systems. The user's task is to ensure they accurately represent their referents, because these are free parameters in the theory (see the discussion below Definition 3.1.9). For this reason, the aim is to demonstrate the *preservation* of these truth systems. This means preserving their axioms and rules of inference. With this goal in mind, we define a **truth predicate** accordingly.

[TODO: clarify how in works! May need to work with a direct representation.]

Definition 5.3.8. Let c be a context. A unit t is called a **truth predicate** for c if for every unit u, $u - c \to t$ if and only if $u - \text{in} \to c$. △

Truth systems in Welkin, however, are separated by contexts. How can it be determined whether a unit is "sound", or truthful about derivations? Our approach is to *merge* the contents and determine if they match. This precisely uses rule ? for whether a representation is contained in a unit. [TODO: still need to clarify global context here! Also, ensure that .]

Definition 5.3.9. A unit u is **sound** if for every context c and representation $\{\{a - d \to b\} - \text{in} \to c\} - \text{in} \to u$, $\{@c, \{a - d \to b\}\} \longleftrightarrow c$. △

In other words, adding the representation $\{a - d \to b\}$ does **not** add new information to c.

[TODO: discuss Tarski's undefinability theorem here.]

In this setting, Gödel's incompleteness theorems manifest in the undecidability of derivations. The second cannot be avoided with soundness outright, but it can with a weaker notion. With the *usual* notions of soundness, the second cannot be avoided. But it this can be mitigated using a weaker notion of soundness.

[TODO: define the equivalent of total computable functions for units! And determine if a **single** input is enough to trust soundness. If not, need to construct examples to show why!]

Definition 5.3.10. A unit u is **serial-sound** if there is a total computable function such that the following property holds: given an arbitrary, but fixed, unit v as input, verifies that any representation "claimed in u" is "actually in v". Moreover the selector must be proven **constructively**, or "without explosion"[TODO: define in general!]. △

Note that not every total computable function can be proven to be total. For this reason, we have a more technical definition in Section 5.4. For now, we can prove the following.

[TODO: make this much more succinct and do not rely on bootstrap anymore] [TODO[SMALL]: make sure to clarify this proof!]

Lemma 5.3.11. *The unit* verifier *is serial-sound.*

Moreover, serial-soundness is transitive, just as with soundness. This corresponds to composition of meta-proofs.

Lemma 5.3.12. *Let u, u', u'' be units, and suppose u is serial-sound. If u proves that u' is serial-sound and u' proves that u'' is serial sound, then u proves that u'' is serial sound.*

DRAFT

Proof. Suppose u is serial-sound. Then, in the context u, u contains the a selector of u'. This selector can then be composed with that from u'' proven in u' to complete the proof. □

5.4. **The Meta Unit.**

[TODO: complete! Want to ensure that proofs can be *chains* of serial-sound units. Moreover, should include *any* expressible derivation that is "reliable"!]

Definition 5.4.13. The **meta recursor** meta over all units is defined recursively:

- **Base case:** $\mathrm{verifier} - \mathrm{in} \to \mathrm{meta}$.
- **Recursive step:** for each proof in meta, adding a self-verifying unit is a valid proof step.

△

If meta verifies a claim C, then we say that meta **meta-proves** C.

Remark 5.4.14. A different approach to create more powerful chains of theories is *reflection*. One example is from Feferman [Fef62]: starting from PA, one can add the encoded statement, roughly meaning: "PA is sound". One can then iterate this through transfinite induction. While this, too, is a way to express all proofs, reflection is a *subset* of our techniques. △

[TODO: make this constructive! Probably possible. And maybe instead of consistency we want *coherency*, to generalize having a false operator? So to prevent *all* things from being proved, or only useful things being proved?]

Theorem 5.4.15. *The unit* meta *meta-proves that serial soundness implies soundness.*

Proof. Let u be a serial sound unit. By way of contradiction, suppose u is not sound. Then there exists a specific instance φ that u contains but is not true. However, as u is finite, there is a finite trace that verifies the failure of soundness. But, because the serial soundness proof is constructive, it reliable proves that *no such* trace exists, a contradiction. Therefore, u must be sound. □

Remarkably, meta *also* meta-proves its own serial soundness.

Corollary 5.4.16. *The unit* meta *meta-proves its own serial soundness, and hence, soundness.*

Proof. Fix some claim C, and suppose C can be proved by a terminating sequence of self-verifying theories. On each C with a proof, define s to be one of these proofs. [TODO: make sure choice is not needed! Probably need a way to standardize this, though maybe it's already in the constructive definition? Or the existence of *any* of these is enough]. Clearly s is total as the sequence terminates, and is correct by construction. This proves meta is serial-sound. Thus, by Theorem 5.4.12, meta is sound □

5.5. **Proof of Complete Expressivity.**

[TODO: find a simpler proof! Maybe we don't *need* ordinals?]

To show that this covers *every* proof that can be computably recognized, we need to discuss recursive ordinals. We provide an embedding via unit. For more background

DRAFT

on ordinals, refer to [Mon73], [Kle38]. To keep the presentation self-contained, we will define a unit for recursive ordinals. [TODO: maybe use a separate appendix?]

At a high level, it is not enough to carry out a *successor* step, and then solely combine theories through a union in the limit stage. At most, we can only reach the Feferman–Schütte ordinal, denoted Γ_0. [CITE!]. [TODO: discuss predicative and impredicative theories more closely!] Feferman himself *advocated* for the sole use of predicative mathematics [CITE]. However, the aim of Welkin is to be *as* expressive as possible, especially as impredicative theories have use in certain areas, e.g., formal methods projects [TODO: cite these!]

Due to Artemov, there is a *massive* jump from $I\Delta_0$ all the way to ZFC. This is because, over $I\Delta_0$ as the base-theory, ZFC is self-verifying (proves its own soundness. A reason for this power in ZFC is the axiom of replacement, or even comprehension. We extend this realization using hyperarithmetic sets, which are known to cover every recursive ordinal, see [Kle38] [TODO: make sure this citation is correct!]. Based on Kleene's theorem, our key construction in the limit case is to add this: let λ be a limit ordinal and consider all given theories $T_1, T_2, ..., T_\beta, ...$ with $\beta < \lambda$. Then $T_\lambda = \bigcup_{\beta<\lambda} T_\beta \cup \mathrm{Comp}_\lambda(\Delta_1^1)$. The set $\mathrm{Comp}_\lambda(\Delta_1^1)$ is an axiom schema over each proposition φ definable in the extended theory, stating that [TODO: clarify notation! And double check hyperarithmetic literature!]

$\exists X_{\varphi,\lambda}.\forall n.\big(n \in X \Leftrightarrow \big(\exists p.\ \mathrm{Prov}\big(T_{\{\psi(p)\}}\big), \varphi(p)\big)\big)$.

Note that this set is encoded through $I\Delta_0$ through an appropriate encoding. And the provability predicate is also encoded through the base theory. [TODO: clarify these encodings!] This new axiom naturally extends the Axiom of Comprehension through *other* sound theories.

[TODO: clarify that hyperarithmetic sets are connected to definability in Δ_1^1!]

We now prove a major property of Welkin.[TODO: definitely clean up!]

Theorem 5.5.17. *The unit* meta *expresses any computably expressible proof. More precisely, for any recursive ordinal* α*, there is a unit* $T \in \mathrm{meta}$ *with greater theoretic proof ordinal than* α*.*

Proof. Using only autonomous progressions [CITE!], it is already know that one can reach the Feferman-Schütte ordinal Γ_0. We may therefore focus on reaching higher ordinals, which depends on the limit case. But [Kle38] already proves that using hyperarithmetic sets in the successor ensures that we can find *some* $\beta > \alpha$. This completes the proof. □

Note, however, that detecting if an ordinal is recursive is undecidable. The upper bound is the best one can hope for, in general.

As an immediate corollary, this proves that unit, being a simple verifier, does *not* impose further restrictions on proofs. Moreover, additional layers of meta-theories are not needed, thanks to how expressive serial-soundness is.

Remark 5.5.18. The definition of information encompasses *every* meta-proof. Theorem 5.5.14 is to show that even *one* chain of theories is enough to majorize each recursive ordinal. And the construction is entirely definable by a Turing machine! △

DRAFT

6. Conclusion

[TODO: update conclusion.]

This thesis introduced Welkin, a universal, formalized information language. The foundations were developed rigorousl. Moreovoer, the syntax (Section 4) was defined with invertible operations, simultaneously defining *parsing* and *printing*. The main grammar was proven to be unambiguous through a suitable transformation (Section 4.6). Finally, Section 5 proved that the definition of information is complete, thereby completing Goal 1 [TODO: add link to goal]. dthe to be accepted by an LL(1) grammar, showing that parsing is unambiguous. The semantics (Section 3) were provided with several passes to convert parse trees into units, which contain both a hierarchical and relational structure for scoping and direct representations, respectively. Units have key properties that enable them to express any partial computable function Theorem 3.3.1, in conjungtion with expressing any truth management system, demonstrates **universality** of the system. This is practically demonstrated by showing that all the major paradigms in Information Management and Knowledge Management are expressible within Welkin. Finally, the bootstrap in Section 3 self-hosts the language within a bounded 64 variant, whose complete Unambiguity (as well as the grammar's prior) establishes **standardization**. Revisions further enhance this by

The remaining sections show several areas for future work. This list is not exhaustive and, by the previous arguments, and can be applied to *any* subject with computable representations (essentially, any human subject).

6.1. Comparisons to Existing Literature.

In addition to domain specific proposals, there are approaches for general IM which still fail to resolve all issues. One prominent example is Burgin's General Theory of Information [Bur09] that comprehensively includes many separate areas for IM, including the complexity-based Algorithmic Information Theory, through a free parameter called an "infological system", which encompasses domain specific terminology and concepts. In contrast to other approaches, Burgin's generalized theory is flexible and enables greater coverage of different kinds of information [Mik23]. Despite this coverage Burgin does not closely tie the free parameter with his formal analysis of Algorithmic Information Theory, making it unclear how to use this in a practical implementation. Burgin's Theory of Information, along with broad proposals, have severe shortcomings, highlighting major obstacles for IM.

6.2. Future Work.

The author has several future projects for Welkin in mind. These include:

- **Writing the Standard:** this would be a self contained document that describes. The Standard would *bootstrap* the language, providing step-by-step instructions on how to implement a Welkin information base. There would also be a core library that enables user created languages that convert *into* Welkin. For example, writing *in* Welkin, a user could define a grammar that takes their own syntax and *converts* it into units.
- **Revision System:** . These would be similar to a belief revision system.
- **Package Managmeent:** building on revision systems, this would .
- **Creating Query Solvers:** this would involve incorporating an information base with a program called a *solver*. Given a query, the solver would first

DRAFT

confirms if there is enough information in the database. If not, the solver runs its own internal methods and heuristics. Moreover, the solvers *themselves* can be specified in Welkin, making them easier to study and maintain.

DRAFT

References

[Ack89] Ackoff, R L, From Data to Wisdom, Journal of Applied Systems Analysis **16** (1989) 3–9.

[Aho+06] Aho, Alfred V., Lam, Monica S., Sethi, Ravi, and Ullman, Jeffrey D., *Compilers: Principles, Techniques, and Tools (2nd Edition)*, Addison Wesley, 2006.

[Ang18] Angles, Renzo, The Property Graph Database Model., in *AMW* (Olteanu, Dan and Poblete, Barbara (eds.), CEUR-WS.org.

[AGH05] Arnold, Ken, Gosling, James, and Holmes, David, *The Java Programming Language*, 4th ed., The Java Series, Addison-Wesley Professional, Upper Saddle River, NJ, 2005.

[Art24] Artemov, Sergei, Serial properties, selector proofs and the provability of consistency, Journal of Logic and Computation **35** no. 3 (2024) exae34.

[Atk23] Atkin, Albert, Peirce's Theory of Signs, Spring2023 ed., Metaphysics Research Lab, Stanford University, 2023.

[Bat00] Bateson, Gregory, *Steps to an ecology of mind*, University of Chicago Press ed., University of Chicago Press, Chicago, 2000.

[Buc91] Buckland, Michael K., Information as thing, Journal of the American Society for Information Science **42** no. 5 (1991) 351–360.

[Bur09] Burgin, Mark, *Theory of Information*, World Scientific, 2009.

[BF95] Buvač, Saša, and Fikes, Richard, A declarative formalization of knowledge translation, in *Proceedings of the Fourth International Conference on Information and Knowledge Management*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 340–347.

[Car20] Carneiro, Mario, Metamath Zero: Designing a Theorem Prover Prover, Springer-Verlag, Bertinoro, Italy, pp. 71–88.

[Cur30] Curry, H. B., Grundlagen der Kombinatorischen Logik, American Journal of Mathematics **52** no. 3 (1930) 509–536.

[Dic25] Dictionary, Oxford English, welkin, n., Oxford University Press, 2025.

[Fef62] Feferman, Solomon, Transfinite Recursive Progressions of Axiomatic Theories, The Journal of Symbolic Logic **27** no. 3 (1962) 259–316.

[Fin85] Fine, Kit, *Reasoning with Arbitrary Objects*, Blackwell, New York, NY, USA, 1985.

[Flo10] Floridi, Luciano, *Information : a very short introduction*, Oxford University Press, Oxford ; New York, 2010.

[GWW07] Goczyla, Krzysztof, Waloszek, Wojciech, and Waloszek, Aleksander, Contextualization of a DL Knowledge Base., p. .

[Göd06] Gödel, Kurt, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I, Monatshefte für Mathematik **149** no. 1 (2006) 1–29.

[Har90] Harnad, Stevan, The Symbol Grounding Problem, Physica D **42** (1990) 335–346.

[Har07] Harrington, Brian, ASKNet: Automated Semantic Knowledge Network., in *AAAI*, AAAI Press, pp. 1862–1863.

[HS13] Hazarika, Shyamanta, and Sinha, Smriti, Formal concept analysis: current trends and directions, Artificial Intelligence Review **44** (2013) 1–40.

[Hit21] Hitzler, Pascal, A review of the semantic web field, Commun. ACM **64** no. 2 (2021) 76–83.

[Hit+09] Hitzler, Pascal, Krötzsch, Markus, Parsia, Bijan, Patel-Schneider, Peter F., and Rudolph, Sebastian, *OWL 2 Web Ontology Language Primer*, 2009.

[HMU07] Hopcroft, John E., Motwani, Rajeev, and Ullman, Jeffrey D., *Introduction to Automata Theory, Languages, and Computation*, Third, Pearson/Addison Wesley, Boston, 2007.

[HP17] Hájek, Petr, and Pudlák, Pavel, *Metamathematics of First-Order Arithmetic*, Perspectives in Logic, Cambridge University Press, 2017.

[KBp] KBpedia, OpenCyc Resources.

[Kle38] Kleene, S. C., On Notation for Ordinal Numbers, The Journal of Symbolic Logic **3** no. 4 (1938) 150–155.

[LM01] Leijen, Daan, and Meijer, Erik, Parsec: Direct Style Monadic Parser Combinators For The Real World, (2001) .

[Len95] Lenat, Douglas B, CYC: A large-scale investment in knowledge infrastructure, Communications of the ACM **38** no. 11 (1995) 33–38.

DRAFT

[Len+90] Lenat, Douglas B., Guha, R. V., Pittman, Karen, Pratt, Dexter, and Shepherd, Mary, Cyc: toward programs with common sense, Commun. ACM **33** no. 8 (1990) 30–49.

[McC93] McCarthy, John, Notes on formalizing context, in *Proceedings of the 13th International Joint Conference on Artifical Intelligence - Volume 1*, Morgan Kaufmann Publishers Inc., Chambery, France, pp. 555–560.

[Men09] Mendelson, Elliott, *Introduction to Mathematical Logic*, 5th ed., Chapman & Hall/CRC, 2009.

[Mes+15] Mesgari, Mostafa, Okoli, Chitu, Mehdi, Mohamad, Nielsen, Finn Årup, and Lanamäki, Arto, ``The sum of all human knowledge'': A systematic review of scholarly research on the content of Wikipedia, Journal of the Association for Information Science and Technology **66** no. 2 (2015) 219–245.

[Mik23] Mikkilineni, Rao, Mark Burgin's Legacy: The General Theory of Information, the Digital Genome, and the Future of Machine Intelligence, Philosophies **8** no. 6 (2023).

[Mon73] Monk, J. Donald, Introduction to Set Theory, Journal of Symbolic Logic **38** no. 1 (1973) 151.

[MDV26] Mora, Angel, Dominguez, Jesus, and Villalon, Ana, fcaR: Formal Concept Analysis, 2026.

[PW81] Paris, Jeff B., and Wilkie, Alex J., \Delta_0 Sets and Induction, in *Proceedings of Open Days in Model Theory and Set Theory* (Guzicki, W., Marek, W., Plec, A., and Rauszer, C. (eds.), Leeds University Press, Jadwisin, Poland, pp. 237–248.

[Rei17] Reisman, Miriam, EHRs: The Challenge of Making Electronic Data Usable and Interoperable, P & T : a peer-reviewed journal for formulary management **42** (2017) 572–575.

[RO10] Rendel, Tillmann, and Ostermann, Klaus, Invertible syntax descriptions: unifying parsing and pretty printing, in *Proceedings of the Third ACM Haskell Symposium on Haskell*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 1–12.

[RWE15] Robinson, Ian, Webber, Jim, and Eifrem, Emil, *Graph Databases*, 2nd ed., O'Reilly Media, Inc., 2015.

[RS70] Rosenkrantz, D.J., and Stearns, R.E., Properties of deterministic top-down grammars, Information and Control **17** no. 3 (1970) 226–256.

[Sch24] Schönfinkel, M., Über die Bausteine der mathematischen Logik, Mathematische Annalen **92** (1924) 305–316.

[Sip13] Sipser, Michael, *Introduction to the Theory of Computation*, Third, Course Technology, Boston, MA, 2013.

[SL17] Smith, Brent, and Linden, Greg, Two Decades of Recommender Systems at Amazon.com, IEEE Internet Computing **21** no. 3 (2017) 12–18.

[Str13] Stroustrup, Bjarne, *The C++ Programming Language*, 4th ed., Addison-Wesley Professional, Boston, MA, 2013.

[Tar83] Tarski, Alfred, The Concept of Truth in Formalized Languages, Hackett Publishing Co., Indianapolis, IN, 1983.

[Tra50] Trakhtenbrot, Boris Abramovich, The impossibility of an algorithm for the decidability problem on finite classes, Dokl.\ Akad.\ Nok.\ SSSR **70** no. 4 (1950) 569–572.

[Tur36] Turing, Alan M., On Computable Numbers, with an Application to the Entscheidungsproblem, Proceedings of the London Mathematical Society **2** no. 42 (1936) 230–265.

[Tur79] Turner, D. A., A new implementation technique for applicative languages, Software: Practice and Experience **9** no. 1 (1979) 31–49.

[Zhu+23] Zhu, Xixi, Liu, Bin, Zhu, Cheng, Ding, Zhaoyun, and Yao, Li, Approximate Reasoning for Large-Scale ABox in OWL DL Based on Neural-Symbolic Learning, Mathematics **11** no. 3 (2023).

Department of Mathematics, University of Colorado at Boulder, Boulder, CO
Email address: oscar-bender-stone@protonmail.com

DRAFT